Failure (an endless loop, and a real pause/resume) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints, covering both halves of this exercise — the endless loop, and the checkpoint/interrupt/resume cycle. Work through them in order, and don't jump ahead until you've genuinely tried both halves. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

Part 1: `graph.invoke({...}, config={"recursion_limit": 5})` on a graph with no real exit raises `GraphRecursionError` once it hits the limit — this is the graph-level version of the step limit Doc07's `while` loop needed too. The point isn't to avoid ever seeing this error; it's to have actually watched it happen once, on purpose, so an accidental version of it in a real project isn't a surprise.

Part 2: the exact pieces:

- `checkpointer = MemorySaver()`, then `graph = builder.compile(checkpointer=checkpointer)`.
- A `thread_id`, e.g. `config = {"configurable": {"thread_id": "run-1"}}` — every `invoke()` using this same `config` reads and writes the *same* saved state.
- Inside one node: `from langgraph.types import interrupt` then `decision = interrupt({"question": "Approve this action?"})` — calling this pauses the graph right there and surfaces the payload you passed it to whoever's running the graph.
- First call: `result = graph.invoke({...}, config=config)` — this returns with the graph paused; `result` carries the interrupt payload (check `result["__interrupt__"]`), not a final answer yet.
- Resume call: `graph.invoke(Command(resume=True), config=config)` — same `config`, same `thread_id`, telling the paused `interrupt()` call what value to "return" so the node can keep going from exactly where it stopped.

Think about what a checkpointer is actually persisting, and when. It's not just "the current value of one field" — it's the *entire* state, plus enough information for LangGraph to know exactly which node to run next. That's why a resumed run doesn't restart the node that called `interrupt()` from

scratch with a blank state — it picks the *same* node back up, with everything that node's earlier code already did still intact, as if nothing had happened in between.

The real design question isn't just "does the state survive" — it's "would the *same* checkpointer setup actually survive your program restarting, not just a second function call in the same script?"

The extra piece that matters here:

- `MemorySaver()` only ever lives in your program's memory — it's genuinely gone the instant the Python process exits, exactly like Doc04's chatbot list. Proving pause/resume "works" by calling `invoke()` twice in the same script, with the same `MemorySaver` object still sitting in a variable, doesn't actually prove state survives a real restart — it only proves the object in memory still has it. A real production checkpointer (`langgraph.checkpoint.sqlite`, `langgraph.checkpoint.postgres`, etc.) writes to an actual database, so the *second* `invoke()` call could happen from an entirely separate program run, hours later, and still resume correctly.
- To genuinely test this like a restart, structure your resume as **a second, separate script run** (or at minimum, delete and recreate every Python variable except the `thread_id` between your "pause" and "resume" code) — if you resume using the exact same in-memory `checkpointer` object your first call already had a reference to, you haven't actually tested that the *state* persisted, only that the *object* did.

Hint 2 — The plan, and almost the whole thing

Part 1, complete:

```
# loop_limit_interrupt_practice.py
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.errors import GraphRecursionError

class LoopState(TypedDict):
    count: int

def node_a(state: LoopState) -> dict:
    return {"count": state["count"] + 1}

def node_b(state: LoopState) -> dict:
    return {"count": state["count"] + 1}

builder = StateGraph(LoopState)
builder.add_node("node_a", node_a)
builder.add_node("node_b", node_b)
builder.add_edge(START, "node_a")
```

```
builder.add_edge("node_a", "node_b")
builder.add_edge("node_b", "node_a")

graph = builder.compile()

try:
    graph.invoke({"count": 0}, config={"recursion_limit": 5})
except GraphRecursionError as e:
    print(f"Stopped on purpose: {e}")
```

Part 2, the plan for the rest:

```
from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import interrupt, Command

class ApprovalState(TypedDict):
    task: str
    approved: bool

def do_task(state):
    decision = interrupt({"question": f"Approve running: {state['task']}?"})
    return {"approved": decision}

builder = StateGraph(ApprovalState)
builder.add_node("do_task", do_task)
builder.add_edge(START, "do_task")
builder.add_edge("do_task", END)

checkpointer = MemorySaver()
graph = builder.compile(checkpointer=checkpointer)

config = {"configurable": {"thread_id": "run-1"}}
starting_state = {"task": "send an email", "approved": False}
first_result = graph.invoke(starting_state, config=config)
print(first_result)  # paused -- look for the interrupt payload here

second_result = graph.invoke(Command(resume=True), config=config)
print(second_result)  # resumed -- approved should now be True
```

Run this and check: does `first_result` actually contain the question you passed to `interrupt()`? Does `second_result["approved"]` come back `True`? Compare against the Solution.

Once that's working, prove it actually survives a real restart, not just a second call in the same script:

```
same Part 2 graph and checkpointer setup, but split into 2 actual
separate script runs sharing only the thread_id (written down or
hardcoded), to prove the state really persists and isn't just
living in a Python variable your script still happens to hold:

pause_script.py:
    build the graph, compile with MemorySaver, invoke once with a
    fixed thread_id, then exit

resume_script.py (run separately, after pause_script.py has exited):
    build the SAME graph shape again (new MemorySaver() object --
    but see the catch below), invoke with Command(resume=...) and
    the SAME thread_id, confirm it resumes correctly
```

Here's the catch worth noticing yourself before reading the Solution: with `MemorySaver`, a genuinely fresh `resume_script.py` process **can't** actually resume — because `MemorySaver`'s data really does live only in the process that created it, and a new process gets a brand-new, empty one. Try it and watch it fail (or simply not find the paused state) — that failure *is* the lesson: it's the proof that `MemorySaver` is exactly as fragile as Doc04's in-memory chatbot list, and that a checkpointer meant to survive a real restart needs to write to an actual file or database (`langgraph.checkpoint.sqlite.SqliteSaver`, for instance), not RAM. For the fix, put the graph in its own module (`approval_graph.py`) and import it from both scripts; in the second script, `graph.get_state(config).next` tells you which node is waiting before you resume.

Full solution: Show me the solution